

FUNTUSH BACKEND

3-Week Development Plan

Git Flow • GitHub Workflow • Daily Tasks

Project	Funtush (TrekNepal)
Team Size	5 Developers
Duration	3 Weeks (18 Working Days)
Hours	Flexible — No 9-5 Requirement
Stack	Node.js + TypeScript + PostgreSQL
Repository	github.com/intersectinfodevelopers/funtush-frontend

OVERVIEW

Flexible Working Hours

You do not have to work 9 AM to 5 PM. Work whenever you want. The only rules are:

- Complete your daily tasks before you sleep
- Push your code to GitHub at the end of every session
- Be available for the daily standup (time agreed by the team)
- If you are blocked for 30 minutes, ask for help immediately

Three Weeks Plan

Week	Focus	Goal
Week 1	Foundation & Database	Everything set up, all developers can build

Week 2	Agency Operating System	Agencies can manage treks, guides, bookings
Week 3	Marketplace & Safety	Public search, reviews, admin panel live

Team Overview

Developer	Role	Branch (Week 1)
Dipesh Singh	Monorepo & Database	feature/ds/monorepo-setup
Prathiva Sharma	Authentication	feature/ps/authentication
Dipika KC	Agency Registration	feature/dk/agency-registration
Shreedev Sharma	Roles & Storage	feature/ss/roles-storage
Sumit Tiruwa	Admin System	feature/st/admin-system

GIT WORKFLOW

Branch Structure

main ← staging ← development ← feature/*

Branch	Purpose	Approvals Needed	Merge Strategy
main	Production	2 approvals + CI pass	Merge commit
staging	UAT / Testing	1 approval + CI pass	Squash and merge
development	Integration	1 approval + CI pass	Squash and merge
feature/*	Developer work	No restriction	PR to development

Branch Naming

Developer	Initials	Example Branch
Dipesh Singh	ds	feature/ds/monorepo-setup
Prathiva Sharma	ps	feature/ps/authentication
Dipika KC	dk	feature/dk/agency-registration
Shreedev Sharma	ss	feature/ss/roles-storage
Sumit Tiruwa	st	feature/st/admin-system

Daily Git Commands

GOLDEN RULE: Always Pull Before Push

```
git fetch upstream && git rebase upstream/development

git fetch upstream
git rebase upstream/development
git add .
git commit -m "feat: what you did"
git push origin feature/your-initials/your-task
```

Friday: Create Pull Request

- Go to GitHub repository
- Click New Pull Request
- Base: development ← Compare: feature/your-branch
- Fill the PR template, add labels, select reviewer
- Click Create Pull Request

Commit Message Format

```
feat: what you added
fix: what you fixed
test: what tests you added
refactor: code cleanup
docs: documentation update
```

WEEK 1 — FOUNDATION & DATABASE

Goal: Build the base. All developers can run the project and connect to database by end of week.

Week 1 Dependencies

Produced By	What	Needed By	Deadline
Dipesh	Monorepo + Database	Everyone	Day 1 EOD
Prathiva	JWT Middleware	Dipika, Shreedev, Sumit	Day 2 EOD
Sumit	Tenant Middleware	Dipika, Prathiva	Day 2 EOD
Shreedev	Permissions Constants	All	Day 3 EOD

Dipesh Singh

Role: Monorepo & Database Setup **Branch:** `feature/ds/monorepo-setup`

Summary: Dipesh must finish Day 1 before others can start. Everything depends on the monorepo and database.

Day	Task	Deliverable
Day 1	Create monorepo folder structure and install all tools	pnpm install works, server starts
Day 2	Design PostgreSQL schema and write Prisma migrations	Database tables created, seed data inserted
Day 3	Set up Redis for sessions, caching, and rate limiting	Redis connected, helper functions exported
Day 4	Health check endpoint and shared utility functions	GET /health returns status for all services
Day 5	Integration tests for database, Redis, and utilities	All tests pass, zero errors
Day 6	Submit PR, address review comments, merge to development	Code merged, team notified

DAY 1 Monorepo Structure

- Create apps/api/ (Express server), packages/database/, packages/utils/, packages/types/, packages/config/
- Set up pnpm-workspace.yaml and turbo.json for build order
- Install: Express, TypeScript, Prisma, PostgreSQL, Redis, Jest
- Create .env.example with all required variables: DATABASE_URL, REDIS_URL, MONGO_URL, JWT_SECRET

Deliverable: pnpm install completes with zero errors

DAY 2 PostgreSQL Schema

- Create Agency table: id, name, email, slug, status, tier, trial_expires_at, created_at
- Create AgencyUser table: id, tenant_id, agency_id, email, password_hash, role
- Create RefreshToken table: id, user_id, token_hash, expires_at
- Create SubscriptionTier table: name, max_staff, max_guides, monthly_price, features
- Run prisma migrate dev and write seed.ts to create Super Admin and test tiers

Deliverable: Database ready, seed data inserted

DAY 3 Redis Setup

- Connect Redis using ioredis with environment variables
- Create helper: setCache(key, value, ttl), getCache(key), deleteCache(key)
- Create rate limit helper: checkRateLimit(ip, limit, window)
- Create session helpers: setSession(userId, data), getSession(userId)

Deliverable: Redis helpers exported, connection verified

DAY 4 Shared Utilities

- Create response.ts: success(data), error(message, code), paginated(data, meta)

- Create pagination.ts: paginate(page, limit) returns { skip, take }
- Create GET /health endpoint: checks DB, Redis, MongoDB, returns status
- Create permissions.ts constants: BOOKING_VIEW, BOOKING_CREATE, GUIDE_MANAGE, FINANCE_VIEW, etc.

Deliverable: GET /health returns: { status: ok, db: ok, redis: ok }

DAY 5 Integration Testing

- Write tests: DB connection, Redis connection, health endpoint, all utility functions
- Run pnpm lint, pnpm type-check, pnpm test — all must pass
- Fix any errors found during testing
- Update README with setup instructions

Deliverable: All tests pass, zero lint errors

DAY 6 Submit and Merge

- Create PR: title 'feat: monorepo setup and database — Week 1'
- List all deliverables in PR description, add label type: feature
- Request review from 1 team member, address feedback
- Merge using Squash and merge, delete feature branch
- Post in Slack: COMPLETED — Week 1 — Dipesh Singh

Deliverable: Merged to development

Prathiva Sharma

Role: Authentication & JWT **Branch:** [feature/ps/authentication](#)

Day	Task	Deliverable
Day 1	Create auth package structure, JWT functions, password hashing	hashPassword, generateAccessToken exported
Day 2	OTP generation, email verification, Express middleware	OTP system works, requireAuth middleware ready
Day 3	Login endpoints for admin, agency, and trekker	POST /auth/login returns JWT tokens
Day 4	Token refresh endpoint and account lockout after 5 attempts	Refresh works, lockout activates
Day 5	Test all auth flows end to end	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Auth Package Setup

- Create packages/auth/ with: jwt.ts, password.ts, otp.ts, middleware.ts, types.ts
- Implement hashPassword(password) using bcrypt with salt rounds 10
- Implement comparePassword(password, hash) returns boolean
- Implement generateAccessToken(payload) — expires 15 minutes
- Implement generateRefreshToken(userId) — expires 7 days
- Implement verifyAccessToken(token) — returns decoded payload or throws

Deliverable: Auth package compiles, all functions exported

DAY 2 OTP and Middleware

- Implement generateOTP() — returns random 6-digit string
- Implement storeOTP(email, otp) — saves to Redis with 15-minute TTL
- Implement verifyOTP(email, otp) — checks Redis, deletes after use
- Implement requireAuth middleware — checks Bearer token, sets req.user
- Implement requireRole(roles[]) middleware — checks role from JWT
- Implement requirePermission(permission) middleware — checks permissions array

Deliverable: OTP tests pass, middleware exports correctly

DAY 3 Login Endpoints

- POST /auth/admin/login — find Super Admin, check password, return tokens
- POST /auth/agency/login — find agency admin, check agency status (not LOCKED/SUSPENDED), return tokens
- POST /auth/trekker/login — find trekker, return tokens
- POST /auth/register — create trekker, send OTP email
- POST /auth/verify-otp — verify code, mark email as confirmed
- Store refresh tokens in DB (RefreshToken table)

Deliverable: All login endpoints return access + refresh tokens

DAY 4 Token Refresh and Lockout

- POST /auth/refresh — verify refresh token, delete old, create new pair
- POST /auth/logout — delete refresh token from DB
- GET /auth/me — return current user from JWT (works for all 3 types)
- Account lockout: after 5 wrong passwords, lock for 15 minutes using Redis counter
- POST /auth/trekker/resend-otp — rate limited to 3 per hour

Deliverable: Refresh works, lockout activates after 5 failed attempts

DAY 5 Testing

- Test correct login returns tokens with correct payload
- Test wrong password returns 401
- Test 5 failed attempts locks account for 15 minutes
- Test expired access token returns 401
- Test refresh token rotation — old token invalid after use
- Test logout deletes refresh token
- Update Postman collection with all endpoints

Deliverable: All tests pass, Postman collection updated

DAY 6 Submit and Merge

- Create PR with description of all endpoints built
- Request review, address feedback
- Merge and post in Slack: COMPLETED — Week 1 — Prathiva Sharma

Deliverable: Merged to development

Dipika KC

Role: Agency Registration **Branch:** `feature/dk/agency-registration`

Day	Task	Deliverable
Day 1	Agency registration endpoint and validation	POST /agencies/register creates account
Day 2	Subscription tiers, trial setup, trial expiry logic	30-day trial starts on registration
Day 3	Agency profile setup — all editable fields	Agency can update full profile
Day 4	KYC form submission and review status tracking	KYC submitted, status visible in dashboard
Day 5	Test full registration and trial flow	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Registration Endpoint

- POST /agencies/register — accepts: company_name, email, password, phone, country
- Validate: email format, password min 8 chars, phone format
- Check: email not already registered (return 409 if duplicate)
- Create: Agency row, AgencyUser row with role admin, RefreshToken
- Auto-generate subdomain slug from company name (greenvalleytrekking → greenvalley)
- If slug taken, append number (greenvalley2)
- Send welcome email with login credentials

Deliverable: POST /agencies/register creates agency and admin user

DAY 2 Subscription and Trial

- GET /subscription-tiers — return all 4 tiers from database (not hardcoded)
- Set trial_expires_at = now + 30 days on registration
- Create cron job (runs daily midnight): check all agencies with trial_expires_at < now
- If trial expired: set status = LOCKED, send warning email
- Locked agencies: can view dashboard but cannot accept bookings or publish packages
- POST /agencies/me/subscribe — process payment, set status = ACTIVE, set tier

Deliverable: 30-day trial starts, daily cron runs at midnight

DAY 3 Agency Profile

- PATCH /agencies/me/profile — update: logo, description, phone[], email[], address, regions
- Each field has show_on_website toggle (true/false)
- Store multiple phone numbers and email addresses as JSON arrays
- Address saves to DB, Google Maps embed renders automatically from address
- PATCH /agencies/me/domain — paid tiers only: set custom domain, return DNS instructions

Deliverable: Agency can update and save complete profile

DAY 4 KYC Submission

- POST /agencies/me/kyc — upload: business_registration, pan_certificate, tourism_license, bank_details
- Store documents in S3, save URLs to KYCSubmission table

- Status flow: SUBMITTED → UNDER_REVIEW → APPROVED / REJECTED
 - GET /agencies/me/kyc — agency sees their current KYC status and rejection reason
 - PATCH /admin/agencies/:id/kyc/approve — Super Admin approves, badge awarded
 - PATCH /admin/agencies/:id/kyc/reject — Super Admin rejects with reason, agency notified
- Deliverable:** KYC submitted, visible to admin for review

DAY 5 Testing

- Test registration: valid data creates account, duplicate email returns 409
 - Test trial: account created with 30-day expiry, cron locks expired trials
 - Test profile: update and retrieve all fields
 - Test KYC: submission, admin approve, admin reject with reason
- Deliverable:** All tests pass

DAY 6 Submit and Merge

- Create PR with all endpoints listed
 - Merge and post: COMPLETED — Week 1 — Dipika KC
- Deliverable:** Merged to development

Shreedev Sharma

Role: Roles & File Storage **Branch:** `feature/ss/roles-storage`

Day	Task	Deliverable
Day 1	Dynamic role and permission system	Agency can create custom roles
Day 2	File upload to S3/Cloudflare R2	POST /upload returns CDN URL
Day 3	Staff management with role assignment	Agency can add and manage staff
Day 4	Image optimization pipeline	Multiple sizes generated on upload
Day 5	Test roles, uploads, and staff	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Roles and Permissions

- Create Role table: id, agency_id, name, description
 - Create Permission table: id, key, description (seed with all PERMISSIONS constants)
 - Create RolePermission table: role_id, permission_key (join table)
 - POST /agencies/me/roles — create custom role with name
 - PATCH /agencies/me/roles/:id/permissions — set which permissions this role has
 - GET /agencies/me/roles — list all roles with their permissions
 - DELETE /agencies/me/roles/:id — only if no staff assigned to it
- Deliverable:** Agency can create role, assign permissions, list roles

DAY 2 File Storage

- Connect to S3 (or Cloudflare R2) using AWS SDK
- POST /upload — accept file, validate MIME type (jpg/png/webp/pdf only), validate max size 10MB

- Generate unique filename using UUID to prevent collisions
- Upload to cloud storage, return public CDN URL
- DELETE /upload — delete file from storage by URL
- POST /upload/multiple — accept up to 5 files, return array of URLs

Deliverable: File upload returns CDN URL, invalid files rejected

DAY 3 Staff Management

- Create AgencyStaff table: id, agency_id, user_id, role_id, is_active, invited_at
- POST /agencies/me/staff — create new staff, send invite email with temp password
- GET /agencies/me/staff — list all staff with role and last activity
- PATCH /agencies/me/staff/:id/role — reassign to different role
- DELETE /agencies/me/staff/:id — set is_active = false (not hard delete)
- GET /agencies/me/staff/:id/activity — last 20 audit log entries for this staff

Deliverable: Staff can be added, assigned roles, deactivated

DAY 4 Image Optimization

- On each image upload, generate 4 versions: 320px, 640px, 1280px, 2560px WebP
- Generate 20px blurred placeholder (base64) for instant perceived load
- Generate JPEG fallback at each size for older browsers
- Push all versions to CDN immediately after upload
- Return URL map: { thumbnail, medium, large, original, placeholder }

Deliverable: 4 image sizes + placeholder generated on every upload

DAY 5 Testing

- Test role creation and permission assignment
- Test requirePermission middleware blocks without correct permission
- Test file upload: valid image accepted, oversized rejected, wrong type rejected
- Test multiple file upload returns correct array
- Test staff add, role change, deactivate

Deliverable: All tests pass

DAY 6 Submit and Merge

- Create PR with all endpoints listed
- Merge and post: COMPLETED — Week 1 — Shreedev Sharma

Deliverable: Merged to development

Sumit Tiruwa

Role: Admin System & Tenant Middleware **Branch:** `feature/st/admin-system`

Day	Task	Deliverable
Day 1	Tenant resolution middleware — domain maps to agency	All routes know which agency they belong to
Day 2	Super Admin dashboard overview endpoints	GET /admin/dashboard returns live stats
Day 3	Rate limiting and request logging	Rate limiting prevents abuse

Day 4	KYC queue and agency management for admin	Admin can approve agencies
Day 5	Test tenant isolation and admin endpoints	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Tenant Middleware

- Create resolveTenant middleware: reads Host header, maps domain to tenant_id
- funtush.com → central_platform context (no tenant_id)
- xyz.funtush.io → agency_xyz tenant_id (from subdomain slug lookup)
- www.custom.com → agency tenant_id (from custom domain mapping table)
- admin.funtush.com → platform admin context (IP whitelist check)
- Cache tenant lookup in Redis (5-minute TTL) to avoid repeated DB queries
- Unknown domain → return 404, do not expose internal errors
- Set req.tenantId, req.agencyId on every request before any route handler runs

Deliverable: Every route knows its tenant context

DAY 2 Admin Dashboard

- GET /admin/dashboard: total agencies by tier, total active subscriptions, revenue this month, active treks live count
- GET /admin/agencies: paginated list with filters (tier, status, country, search by name/email)
- GET /admin/agencies/:id: full agency profile, all settings, booking summary, financial summary
- PATCH /admin/agencies/:id/status: ACTIVE / SUSPENDED / LOCKED with reason
- PATCH /admin/agencies/:id/tier: change subscription tier
- POST /admin/agencies/:id/break-glass: generate time-limited emergency access token, notify agency admin immediately

Deliverable: GET /admin/dashboard returns live platform stats

DAY 3 Rate Limiting and Logging

- Create rate limit middleware using Redis: store request count per IP per endpoint
- POST /auth/agency/login: max 5 requests per minute per IP
- POST /auth/trekker/register: max 3 per hour per IP
- General API: max 200 requests per minute per IP
- SOS routes: exempt from rate limiting (safety critical)
- Create request logger: method, path, status code, response time, request ID
- Store logs in MongoDB (not PostgreSQL) for flexible querying

Deliverable: Rate limiting blocks after limit, SOS always passes

DAY 4 KYC Queue and Email Queue

- GET /admin/kyc-queue: all pending KYC submissions sorted by date
- GET /admin/kyc/:id: view full submission with all document URLs
- PATCH /admin/kyc/:id/approve: set status APPROVED, award verification badges
- PATCH /admin/kyc/:id/reject: set status REJECTED, save reason, notify agency
- Create MongoDB EmailQueue model: to, subject, body, status (pending/sent/failed), retry_count
- Create queueEmail(to, subject, body) helper: saves to EmailQueue
- GET /admin/email-queue: pending, sent, and failed emails

Deliverable: Admin can approve or reject KYC, emails queued

DAY 5 Testing

- Test tenant resolution: xyz.funtush.io maps to correct agency
- Test tenant isolation: login as Agency A, verify Agency B data is not accessible
- Test rate limiting: 6th request within 1 minute returns 429
- Test SOS route: never returns 429 regardless of request count
- Test admin dashboard stats are accurate
- Test break-glass: agency notified, access token works, expires on time

Deliverable: Tenant isolation verified, rate limiting works

DAY 6 Submit and Merge

- Create PR with all endpoints and middleware listed
- Merge and post: COMPLETED — Week 1 — Sumit Tiruwa

Deliverable: Merged to development

WEEK 2 — AGENCY OPERATING SYSTEM

Goal: Agencies can now fully run their business — create treks, manage guides, accept bookings, track customers.

Dipesh Singh

Role: Trek Package Builder **Branch:** [feature/ds/trek-packages](#)

Day	Task	Deliverable
Day 1	Package and itinerary database schema	Tables created, migrations run
Day 2	Package creation and editing API	POST /agencies/packages works
Day 3	Itinerary builder — day by day content	Itinerary can be built and reordered
Day 4	Departure dates with slot management	Overbooking prevented
Day 5	Testing package system	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Package Schema

- Create TrekPackage: id, agency_id, title, description, duration_days, price_per_person, difficulty, max_group_size, status (DRAFT/PUBLISHED/ARCHIVED)
- Create TrekDestination: id, agency_id, name, region, altitude_m, best_season
- Create TrekItinerary: id, package_id, day_number, location, description, altitude_m
- Create TrekDepartureDates: id, package_id, start_date, max_slots, booked_slots, status (AVAILABLE/FULL/GUARANTEED)
- Create TrekAddOn: id, package_id, name, price, per_person (boolean)
- Connect TrekPackage to TrekDestination (many destinations per package)

Deliverable: All tables created and linked

DAY 2 Package API

- POST /agencies/packages — create package, validate required fields
- PATCH /agencies/packages/:id — edit package fields
- GET /agencies/packages — list with filters: status, destination
- POST /agencies/packages/:id/publish — validate completeness before allowing publish
- POST /agencies/packages/:id/duplicate — clone package as DRAFT
- DELETE /agencies/packages/:id — set status ARCHIVED (soft delete)

Deliverable: Agency can create, edit, publish, and archive packages

DAY 3 Itinerary Builder

- POST /packages/:id/itinerary — add day with: day_number, location, description, photos[]
- PUT /packages/:id/itinerary/:day — update existing day content
- DELETE /packages/:id/itinerary/:day — remove a day
- PATCH /packages/:id/itinerary/reorder — reorder days by submitting new day_number array

Deliverable: Full itinerary management with reordering

DAY 4 Departure Dates

- POST /packages/:id/dates — add departure date with max_slots
- PATCH /packages/:id/dates/:dateId — update status or slot count
- DELETE /packages/:id/dates/:dateId — remove departure date
- On booking confirmed: booked_slots++ and check: if booked_slots >= max_slots → set status FULL
- Prevent inquiry submission if date status is FULL

Deliverable: Slot management prevents overbooking

DAY 5 Testing

- Test create/edit/publish/archive package
- Test itinerary day CRUD and reorder
- Test departure date management
- Test full slots prevent inquiry

Deliverable: All tests pass

DAY 6 Submit and Merge

- Merge and post: COMPLETED — Week 2 — Dipesh Singh

Deliverable: Merged to development

Prathiva Sharma

Role: Booking Flow **Branch:** `feature/ps/booking-flow`

Day	Task	Deliverable
Day 1	Booking database schema	All booking tables ready
Day 2	Inquiry submission with OTP verification	Trekker can submit inquiry
Day 3	Agency response and payment link generation	Agency can accept or reject

Day 4	Payment webhook handling and booking lifecycle	Payment confirms booking
Day 5	Testing full booking flow	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Booking Schema

- Create Booking: id, agency_id, trekker_id, package_id, departure_date_id, group_size, total_price, status
- Booking status enum: INQUIRY → PENDING → CONFIRMED → PAYMENT_PENDING → PAID → ACTIVE → COMPLETED → CANCELLED
- Create BookingAddOn: booking_id, addon_id, quantity (tracks selected add-ons)
- Create PaymentLink: id, booking_id, url_token, expires_at, used (boolean)

Deliverable: All booking tables created

DAY 2 Inquiry Submission

- POST /bookings/inquiry — trekker selects package, date, group size, add-ons, contact details
- Step 1: send 6-digit OTP to trekker email (stored in Redis, 15-minute expiry)
- Step 2: trekker submits OTP — POST /bookings/inquiry/verify-otp
- Step 3: inquiry saved to DB with status INQUIRY
- Agency admin receives push notification + email: New Inquiry from [trekker name]
- Trekker receives confirmation email: Your inquiry has been submitted

Deliverable: Inquiry submitted after OTP verified

DAY 3 Agency Response

- GET /agencies/me/bookings — list with tabs: INQUIRY, CONFIRMED, ACTIVE, COMPLETED
- PATCH /bookings/:id/accept — agency confirms, generate payment link (48-hour expiry)
- PATCH /bookings/:id/reject — agency rejects with reason, trekker notified
- PATCH /bookings/:id/propose-date — offer alternative date, trekker notified
- Trekker notified by email + push for every status change

Deliverable: Agency can accept/reject/propose from dashboard

DAY 4 Payment Webhook

- Each agency has a unique webhook URL: /webhooks/payment/{agency_id}
- On payment gateway callback: verify signature, confirm amount matches booking total
- Set booking status: PAID, decrement available slot count
- Assign guide to booking (if auto-assignment configured)
- Send to trekker: booking confirmation email with full itinerary PDF attached
- Send to guide: assigned trek notification with trekker details

Deliverable: Payment confirmed, booking status updates automatically

DAY 5 Testing

- Test full flow: inquiry → OTP → agency accept → payment link → webhook → PAID
- Test rejection and date proposal flows
- Test payment link expiry (48 hours)
- Test slot count decrements on payment

Deliverable: All tests pass

DAY 6 Submit and Merge

- Merge and post: COMPLETED — Week 2 — Prathiva Sharma
- Deliverable:** Merged to development

Dipika KC

Role: Customer CRM Branch: feature/dk/customer-crm

Day	Task	Deliverable
Day 1	Trekker profile and identity model	Unified trekker account works across agencies
Day 2	Customer list with filters and search	Agency can find and filter customers
Day 3	Internal notes and communication log	Staff can add notes per customer
Day 4	Customer analytics and repeat visitor tracking	Agency sees full customer history
Day 5	Testing CRM	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Trekker Profile

- Create Trekker: id, email, name, phone, country, emergency_contact_name, emergency_contact_phone
- One Trekker account works across all agencies (unified identity)
- Agency only sees their own bookings with this trekker — not cross-agency history
- Platform sees full history across all agencies
- Create TrekkerPreference: trekker_id, preferred_destinations[], budget_range, group_size_preference

Deliverable: Trekker can log in on any agency site with same credentials

DAY 2 Customer List

- GET /agencies/me/customers — paginated list of all trekkers who booked with this agency
- Filter: repeat customers only, new customers only, by destination, by booking status
- Search: by name, by email, by phone
- Sort: by last booking date, by total bookings, by total spending
- Show repeat_visitor flag if trekker has more than 1 booking with this agency

Deliverable: Customer list with filters, search, sorting

DAY 3 Notes and Communication

- Create CustomerNote: id, agency_id, trekker_id, staff_id, note_text, created_at
- POST /customers/:id/notes — add internal note (private to agency staff)
- GET /customers/:id/notes — list notes, sorted newest first
- Notes are never visible to the trekker, only to agency staff
- Create CommunicationLog: save WhatsApp and email interactions with timestamps

Deliverable: Staff can add and view private notes per customer

DAY 4 Customer Analytics

- GET /customers/:id/profile — full view: contact, booking history, total spent, visit count, notes
- GET /agencies/me/customers/analytics — top customers by spending, repeat rate, new vs returning ratio
- Create loyalty flag: if trekker booked 3+ times → show Loyal Customer badge on profile
- Track: first booking date, last booking date, average booking value, preferred destinations

Deliverable: Full customer intelligence visible to agency

DAY 5 Testing

- Test customer list: filter, search, sort
- Test notes: add, view, confirm trekker cannot see notes
- Test analytics: repeat count, spending total
- Test loyalty flag activates at 3+ bookings

Deliverable: All tests pass

DAY 6 Submit and Merge

- Merge and post: COMPLETED — Week 2 — Dipika KC

Deliverable: Merged to development

Shreedev Sharma

Role: Guide Management **Branch:** `feature/ss/guide-management`

Day	Task	Deliverable
Day 1	Guide profile and certification database	Guide table and cert table created
Day 2	Guide CRUD API with document uploads	Agency can add and manage guides
Day 3	Availability calendar management	Guide cannot be double-booked
Day 4	Certification expiry alerts	Alert 30 days before expiry
Day 5	Testing guide system	All tests pass
Day 6	Submit PR and merge	Merged to development

DAY 1 Guide Schema

- Create Guide: id, agency_id, name, phone, email, photo_url, bio, languages[], status (ACTIVE/INACTIVE)
- Create GuideCertification: id, guide_id, cert_name, cert_number, issuing_body, expiry_date, document_url
- Create GuideAvailability: id, guide_id, date, status (AVAILABLE/UNAVAILABLE/ASSIGNED)
- Create GuideAssignment: id, guide_id, booking_id, trek_date (links guides to bookings)

Deliverable: All guide tables created

DAY 2 Guide API

- POST /guides — create guide with photo upload (uses Shreedev's upload endpoint)
- PATCH /guides/:id — update guide profile fields

- GET /guides — list all guides with status, current assignment, cert status
- DELETE /guides/:id — set status INACTIVE (soft delete)
- POST /guides/:id/certifications — upload cert document, save details
- DELETE /guides/:id/certifications/:certId — remove certification

Deliverable: Full guide CRUD with certification management

DAY 3 Availability Management

- POST /guides/:id/availability — mark dates as AVAILABLE or UNAVAILABLE
- GET /guides/:id/availability?month=2024-03 — return calendar for that month
- On booking assigned: set those dates to ASSIGNED (cannot book again)
- On booking cancelled: revert dates back to AVAILABLE
- Prevent assigning guide to booking if any date of trek is not AVAILABLE

Deliverable: Double-booking prevented by availability system

DAY 4 Certification Alerts

- Cron job runs daily: query all certifications with expiry_date between now and now+30 days
- Send email to agency admin: Guide [name]'s [cert name] expires in [X] days
- Show EXPIRING SOON badge on guide card in dashboard
- Show EXPIRED badge if expiry_date has passed
- GET /guides/expiring-certs — list all guides with certifications expiring in 30 days

Deliverable: Agency sees expiring certs 30 days in advance

DAY 5 Testing

- Test guide create, update, list, deactivate
- Test certification upload and view
- Test availability: set unavailable, try to assign, expect error
- Test expiry alert: manually set expiry to tomorrow, verify alert fires

Deliverable: All tests pass

DAY 6 Submit and Merge

- Merge and post: COMPLETED — Week 2 — Shreedev Sharma

Deliverable: Merged to development

Sumit Tiruwa

Role: Booking Analytics **Branch:** `feature/st/booking-analytics`

Day	Task	Deliverable
Day 1	Analytics event tracking schema	Every action tracked in MongoDB
Day 2	Agency analytics API	Agency sees their booking and revenue stats
Day 3	Admin-level platform analytics	Admin sees platform-wide data
Day 4	Reports generation (PDF and CSV)	Downloadable monthly reports
Day 5	Testing analytics	All tests pass

Day 6	Submit PR and merge	Merged to development
-------	---------------------	-----------------------

DAY 1 Analytics Schema

- Create MongoDB AnalyticsEvent: agency_id, event_type, trekker_id, package_id, timestamp, metadata
- Event types: PAGE_VIEW, INQUIRY_SUBMITTED, BOOKING_CONFIRMED, BOOKING_PAID, BOOKING_CANCELLED
- Track on every booking state change (via middleware — no manual calls needed in routes)
- Create DailyAgencySummary: date, agency_id, new_inquiries, confirmed, revenue, cancelled (updated nightly)

Deliverable: Every booking event tracked automatically

DAY 2 Agency Analytics

- GET /agencies/me/analytics?period=monthly — bookings by day, revenue by day, conversion rate
- GET /agencies/me/analytics/packages — top packages by bookings and revenue
- GET /agencies/me/analytics/customers — new vs returning, top customers, geographic sources
- GET /agencies/me/analytics/guides — guide utilization rate, bookings per guide
- Support periods: last_7_days, last_30_days, last_12_months, custom (from/to dates for Medium and Large tiers)

Deliverable: Agency analytics with charts data

DAY 3 Platform Analytics

- GET /admin/analytics — total bookings platform-wide, revenue by tier, top destinations globally
- GET /admin/analytics/agencies — top performing agencies, highest revenue, best retention
- GET /admin/analytics/marketplace — most searched destinations, popular filters, conversion funnel
- GET /admin/analytics/tiers — trial to paid conversion rate, churn rate per tier

Deliverable: Super Admin sees platform-wide performance

DAY 4 Reports

- GET /agencies/me/reports/monthly?month=2024-03 — PDF report: bookings, revenue, guides, top packages
- GET /agencies/me/reports/annual?year=2024 — annual PDF summary
- Both reports exportable as PDF (using puppeteer or similar) or CSV
- Report cached in S3 for 24 hours (avoid regenerating on every request)

Deliverable: Monthly PDF/CSV reports downloadable

DAY 5 Testing

- Test analytics data accuracy by creating known bookings
- Test period filters work correctly
- Test report generation returns correct file
- Test admin analytics shows correct platform-wide totals

Deliverable: All tests pass

DAY 6 Submit and Merge

- Merge and post: COMPLETED — Week 2 — Sumit Tiruwa

Deliverable: Merged to development

WEEK 3 — MARKETPLACE & FINAL INTEGRATION

Goal: Public marketplace live, reviews working, admin fully functional. Code ready for production deployment.

Dipesh Singh

Role: Central Marketplace **Branch:** `feature/ds/marketplace`

Day	Task	Deliverable
Day 1	Meiliseach index setup for packages and agencies	Search returns results in under 100ms
Day 2	Marketplace search and filter API	Travelers can search all packages
Day 3	Agency listing and destination pages	Agency directory live
Day 4	Featured packages and trending logic	Homepage curated content
Day 5	Testing and performance	Under 100ms search response
Day 6	Submit PR, merge to staging	Code on staging for UAT

DAY 1 Search Index

- Install and configure Meiliseach
- Create package index: searchable fields: title, description, destination, agency_name | filterable: difficulty, price, duration, season, altitude
- Create agency index: searchable: name, description, destinations | filterable: tier, region, rating
- Sync on publish: when agency publishes package, index updates within 5 seconds
- Enable typo tolerance: Anapurna → Annapurna, Everst → Everest

Deliverable: Search index live, typo tolerance enabled

DAY 2 Marketplace Search API

- GET /marketplace/packages?q=everest&difficulty=moderate&price_max=1500 — full text search with filters
- GET /marketplace/packages — no query returns all packages, sorted by relevance score
- Pagination: page and limit params, return meta.total, meta.pages
- Trekker preference personalization: if logged in, boost agencies they booked before
- Tier visibility: Large = highest weight, Medium = medium, Small = low, Free = hidden

Deliverable: Search endpoint with filters and personalization

DAY 3 Agency Directory

- GET /marketplace/agencies — list all agencies with tier, rating, top destination tags
- GET /marketplace/agencies/:slug — public agency profile with packages, reviews, badges
- GET /marketplace/destinations — list all destinations with package count
- GET /marketplace/destinations/:slug — master destination page with all agencies operating there

- Master destination pages managed by Super Admin team, not individual agencies

Deliverable: Agency directory and destination pages live

DAY 4 Featured and Trending

- GET /marketplace/featured — return mix: Large tier sponsored + highest rated + most booked this month
- Sponsored badge on any agency with manual priority override from Super Admin
- GET /marketplace/trending — packages with most inquiries in last 7 days
- GET /marketplace/seasonal — packages where best_season matches current month

Deliverable: Homepage sections return curated content

DAY 5 Performance Testing

- Run load test: 100 concurrent search requests — all under 100ms
- Test typo tolerance: misspelled searches return correct results
- Test visibility algorithm: Free agency does not appear, Large appears higher
- Test personalization: logged-in trekker sees previously booked agencies first

Deliverable: Search under 100ms, all rules verified

DAY 6 Submit PR to Staging

- Merge to development first, then create PR: development → staging
- Post: COMPLETED — Week 3 — Dipesh Singh

Deliverable: Merged to staging

Prathiva Sharma

Role: Visibility Algorithm **Branch:** feature/ps/marketplace-visibility

Day	Task	Deliverable
Day 1	Agency visibility score calculation	Scores calculated for all agencies
Day 2	Marketplace ranking API uses scores	Ranking reflects tier and quality
Day 3	Priority override with Sponsored badge	Admin can boost agencies
Day 4	Impression and click tracking	Agency sees their marketplace performance
Day 5	Testing ranking and analytics	All tests pass
Day 6	Submit PR to staging	Merged to staging

DAY 1 Visibility Score

- Create AgencyVisibilityScore: agency_id, base_score, quality_bonus, admin_override, final_score
- Base score by tier: Large = 100, Medium = 50, Small = 25, Free = 0
- Quality bonus: rating 4.5+ = +20, recent booking (last 7 days) = +5, profile complete = +10
- Admin override: set per-agency, additive on top of base + quality
- Recalculate daily via cron job (not per-request to keep search fast)

Deliverable: All agencies have visibility score, recalculated nightly

DAY 2 Ranking in Search

- Marketplace search sorts by final_score DESC as primary sort
- Trekker loyalty override: if trekker previously booked agency, boost +50 for that trekker only
- Tie-breaking: equal score → sort by average rating DESC, then by recent activity
- PATCH /admin/agencies/:id/visibility — set admin_override value, update final_score immediately
- Sponsored label: if admin_override > 0, return sponsored: true in API response

Deliverable: Rankings match score, sponsored agencies labeled

DAY 3 Priority Override

- Super Admin sets priority_override from admin panel
- API returns sponsored: true for these agencies in marketplace results
- Frontend shows Sponsored badge on listing card
- Priority override is a commercial arrangement — disclosed to trekkers
- Override does not change tier or any features, only marketplace position

Deliverable: Sponsored placement disclosed and labeled

DAY 4 Impression Tracking

- On every marketplace search result: increment impression count per agency per day
- On agency card click: record click event with destination searched
- GET /agencies/me/marketplace/impressions?period=last_30_days — return impressions, clicks, CTR
- GET /agencies/me/marketplace/conversions — impressions that converted to inquiry

Deliverable: Agency can see their marketplace performance

DAY 5 Testing

- Test scores: Large ranks above Medium, Medium above Small, Free absent
- Test loyalty: logged-in trekker sees previously booked agency first
- Test priority override moves agency to top with Sponsored badge
- Test impression and click counts increment correctly

Deliverable: All tests pass

DAY 6 Submit PR to Staging

- Merge to development then staging
- Post: COMPLETED — Week 3 — Prathiva Sharma

Deliverable: Merged to staging

Dipika KC

Role: Review System **Branch:** feature/dk/review-system

Day	Task	Deliverable
Day 1	Review database and submission rules	Only verified bookings can review
Day 2	Review submission and post-trek trigger	Auto-invite 24 hours after trek ends
Day 3	Review display and agency response	Reviews visible publicly
Day 4	Review moderation for Super Admin	Admin can remove policy violations

Day 5	Testing review system	All tests pass
Day 6	Submit PR to staging	Merged to staging

DAY 1 Review Schema

- Create Review: id, booking_id, trekker_id, agency_id, guide_id, rating (1-5), text, photos[], verified (always true)
- One review per booking (enforced by unique constraint on booking_id)
- Review can only be submitted if booking status = COMPLETED
- Create ReviewResponse: id, review_id, agency_admin_id, response_text, responded_at
- Create ReviewFlag: id, review_id, reason, flagged_by, status (PENDING/DISMISSED/REMOVED)

Deliverable: Review tables with one-per-booking constraint

DAY 2 Review Submission

- Cron job: daily check bookings where end_date = yesterday and status = COMPLETED
- Send review invitation email with one-time token link
- POST /reviews — accept rating, text, photos via invitation token
- Validate: token valid, not already used, booking status COMPLETED
- On submit: publish review immediately, notify agency admin

Deliverable: Review auto-invited after trek, published immediately

DAY 3 Review Display and Response

- GET /agencies/:slug/reviews — public reviews sorted by newest, includes average rating
- Show star distribution (5-star: X%, 4-star: Y%, etc.)
- POST /reviews/:id/response — agency admin posts public reply
- Response visible publicly below the review
- Agency cannot delete reviews — only flag for admin review
- POST /reviews/:id/flag — agency flags review with reason, goes to admin moderation queue

Deliverable: Reviews public, agency can respond but not delete

DAY 4 Review Moderation

- GET /admin/reviews/flagged — all flagged reviews with reason and agency that flagged
- PATCH /admin/reviews/:id/remove — delete review if it violates content policy
- PATCH /admin/reviews/:id/dismiss-flag — clear flag, review stays published
- Agency notified of admin decision
- Rule: negative genuine reviews stay. Only policy violations removed.

Deliverable: Admin moderation queue for flagged reviews

DAY 5 Testing

- Test only completed bookings can review (pending booking returns 403)
- Test one review per booking (second attempt returns 409)
- Test agency response visible publicly
- Test flag → admin remove → review deleted
- Test flag → admin dismiss → review stays

Deliverable: All tests pass

DAY 6 Submit PR to Staging

- Merge to development then staging
 - Post: COMPLETED — Week 3 — Dipika KC
- Deliverable:** Merged to staging

Shreedev Sharma

Role: Package Publishing Pipeline **Branch:** [feature/ss/publish-pipeline](#)

Day	Task	Deliverable
Day 1	Package publication states and validation	Publish checks completeness
Day 2	PDF generation from package content	Branded PDF downloadable
Day 3	SEO settings per page	Meta tags and structured data
Day 4	Blog system for Medium and Large tiers	Agencies can publish content
Day 5	Testing publishing pipeline	All tests pass
Day 6	Submit PR to staging	Merged to staging

DAY 1 Publication States

- Package status: DRAFT (agency only), PUBLISHED (live + marketplace), UNLISTED (accessible via link only), ARCHIVED (hidden)
 - POST /packages/:id/publish — check completeness: needs title, description, at least 1 photo, at least 1 departure date, at least 3 itinerary days
 - If incomplete: return 400 with list of missing fields
 - Only PUBLISHED packages appear in marketplace search index
 - PATCH /packages/:id/unlist — move to UNLISTED (keeps URL but removes from marketplace)
- Deliverable:** Publish gate checks completeness before going live

DAY 2 PDF Generation

- On publish: generate PDF from package content using Puppeteer
 - PDF includes: agency logo, package title, itinerary table, cost includes/excludes, packing list, agency contact
 - Store generated PDF in S3, cache URL in package record
 - Regenerate PDF on every content update (async — does not block save)
 - GET /packages/:id/pdf — return PDF URL for download button
 - Filename: package-title-agency-name.pdf (SEO-friendly)
- Deliverable:** Branded PDF generated on publish, regenerated on update

DAY 3 SEO Settings

- PATCH /packages/:id/seo — agency sets: meta_title, meta_description, og_image_url
- Auto-generate if not set: meta_title = package title + agency name, meta_description = first 155 chars of description
- Structured data auto-generated: TouristTrip JSON-LD, FAQPage JSON-LD from FAQ section, Review JSON-LD
- All pages included in sitemap.xml — regenerated on publish/unpublish
- GET /agencies/:slug/sitemap.xml — return sitemap for this agency's white-label site

Deliverable: SEO metadata and structured data on all pages

DAY 4 Blog System

- Medium and Large tiers only — return 403 for Small and Free
- Create BlogPost: id, agency_id, title, slug, content (rich text), status (DRAFT/PUBLISHED/SCHEDULED), published_at
- POST /agencies/me/blog — create post
- PATCH /agencies/me/blog/:id — update post content, status, scheduled time
- GET /agencies/:slug/blog — public blog listing (published posts only)
- GET /agencies/:slug/blog/:postSlug — single post
- Blog posts included in sitemap.xml and generate their own SEO tags

Deliverable: Blog CRUD with schedule publish for Medium/Large tiers

DAY 5 Testing

- Test publish with missing fields returns error listing what is missing
- Test PDF generates correctly on publish
- Test SEO auto-generation when fields not set
- Test sitemap includes all published pages
- Test blog is blocked for Small tier, works for Medium

Deliverable: All tests pass

DAY 6 Submit PR to Staging

- Merge to development then staging
- Post: COMPLETED — Week 3 — Shreedev Sharma

Deliverable: Merged to staging

Sumit Tiruwa

Role: Admin Full Dashboard **Branch:** [feature/st/admin-dashboard](#)

Day	Task	Deliverable
Day 1	Admin agency management — full CRUD	Admin can suspend/upgrade any agency
Day 2	SOS monitoring dashboard	Live safety alerts visible to admin
Day 3	Ad campaign review queue	Admin approves ad campaigns
Day 4	Fraud detection queue	Fraud flags visible and actionable
Day 5	Testing admin dashboard	All tests pass
Day 6	Submit PR to staging + final integration	All code on staging

DAY 1 Agency Management

- GET /admin/agencies — paginated with filters: tier, status, country, search, date joined
- GET /admin/agencies/:id — full profile: settings, staff count, booking count, revenue summary, KYC status
- PATCH /admin/agencies/:id/tier — change subscription tier (takes effect immediately)

- PATCH /admin/agencies/:id/status — ACTIVE / SUSPENDED / LOCKED with mandatory reason field
- POST /admin/agencies/:id/impersonate — generate short-lived token to log in as agency admin for support
- All admin actions written to immutable audit log in MongoDB

Deliverable: Admin can fully manage any agency

DAY 2 SOS Monitoring

- GET /admin/sos/active — live feed of all active SOS incidents across all agencies
- Each incident: agency name, guide name, trekker name, GPS coordinates, time since triggered, agency acknowledgment status
- Response timer: turns red if agency has not acknowledged within 15 minutes
- POST /admin/sos/:id/notes — admin adds observation notes
- GET /admin/sos/history — all past incidents with full timeline and resolution
- POST /admin/agencies/:id/warning — issue formal safety warning (stored permanently)
- GET /admin/sos/:id/export — structured export for law enforcement

Deliverable: Admin sees all live SOS incidents with response timer

DAY 3 Ad Campaign Queue

- GET /admin/ad-campaigns/pending — all pending campaigns from Large tier agencies
- Each item: agency name, generated creative (image URLs, copy text, targeting parameters)
- PATCH /admin/ad-campaigns/:id/approve — push campaign live via Meta and Google APIs
- PATCH /admin/ad-campaigns/:id/reject — send rejection reason back to agency
- GET /admin/ad-campaigns/active — all running campaigns with impressions, clicks, spend
- PATCH /admin/ad-campaigns/:id/pause — pause running campaign immediately

Deliverable: Admin can approve, reject, and monitor all ad campaigns

DAY 4 Fraud Queue

- GET /admin/fraud/queue — all flagged accounts sorted by signal strength (red/orange/yellow)
- Each item: account name, email, flags triggered, registration date, evidence summary
- PATCH /admin/fraud/:id/confirm — permanently ban account, add fingerprint/IP/email to blocklist
- PATCH /admin/fraud/:id/dismiss — clear flag, reset risk profile, notify agency
- GET /admin/fraud/ban-registry — all permanently banned accounts with ban reasons

Deliverable: Fraud review queue with ban/dismiss actions

DAY 5 Testing

- Test agency management: tier change, status change, impersonation
- Test SOS: trigger SOS, verify admin dashboard shows alert
- Test ad campaign: submit campaign, admin approves, verify status change
- Test fraud: flag account, admin confirms, account suspended

Deliverable: All tests pass

DAY 6 Final Integration and Staging

- Submit PR to development, then development → staging
- Run full end-to-end test on staging environment
- Verify: registration → trial → payment → packages → bookings → reviews all work
- Post: COMPLETED — Week 3 — Sumit Tiruwa
- Team: staging is ready for UAT — begin testing

Deliverable: All code on staging, UAT testing begins

END OF 3 WEEKS — PRODUCTION CHECKLIST

Before Merging Staging to Main

- All 5 Week 3 PRs merged to staging
- UAT testing complete — no critical or high bugs
- All CI checks passing on staging branch
- Performance: API responses under 300ms on average
- Health check returns OK for all services
- No hardcoded secrets or credentials in code
- All environment variables documented in .env.example
- README updated with deployment instructions

Final Deployment

- Create PR: staging → main (requires 2 approvals)
- GitHub Actions auto-deploys to Vercel on merge to main
- Monitor error logs for 30 minutes after deployment
- Share production URL with team

What We Built in 3 Weeks

Week 1	Week 2	Week 3
Monorepo and database	Trek package builder	Central marketplace
Authentication (JWT)	Booking flow (OTP → pay)	Visibility algorithm
Agency registration	Customer CRM	Review system
Roles and file storage	Guide management	Package publishing + PDF
Admin system and tenant	Booking analytics	Full admin dashboard + SOS

Funtush is live. Ship it.